

Python Practice Tasks: Loops & Data Structures

Task 1: Fruit Vendor Profit Calculator

Problem Statement: You're a fruit vendor trying to reach a target profit. You have different fruits with varying prices and quantities. Find the optimal combination to sell while maintaining minimum stock for each fruit type.

Data:

python

```
fruits = [
    {"type": "mango", "price": 300, "quantity": 10},
    {"type": "apple", "price": 200, "quantity": 20},
    {"type": "banana", "price": 100, "quantity": 30},
    {"type": "orange", "price": 150, "quantity": 25},
    {"type": "grape", "price": 250, "quantity": 15}
]
target_profit = 10000
min_sell_per_type = 5
```

Requirements:

- Calculate how many fruits of each type to sell to reach target profit
- Ensure minimum 5 fruits sold per type
- Use loops, dictionaries, and conditional statements

Task 2: Student Grade Management System

Problem Statement: A teacher needs to manage student grades across multiple subjects and calculate class statistics.

Data:

python

```
students = {
    "Alice": [85, 90, 78, 92, 88],
    "Bob": [76, 82, 90, 85, 79],
    "Charlie": [92, 95, 89, 96, 93],
    "Diana": [68, 72, 75, 70, 74],
    "Eve": [88, 86, 84, 90, 87]
}
subjects = ["Math", "Science", "English", "History", "Art"]
```

Requirements:

- Calculate average grade for each student
 - Find top performer in each subject
 - Identify students who need improvement (average < 80)
 - Create a grade distribution report
-

Task 3: E-commerce Inventory Manager

Problem Statement: An online store needs to track inventory, process orders, and manage stock levels across multiple warehouses.

Data:

python

```
inventory = {  
    "laptop": {"price": 50000, "stock": {"warehouse_a": 15, "warehouse_b": 8}},  
    "smartphone": {"price": 25000, "stock": {"warehouse_a": 25, "warehouse_b": 12}},  
    "tablet": {"price": 20000, "stock": {"warehouse_a": 10, "warehouse_b": 20}},  
    "headphones": {"price": 3000, "stock": {"warehouse_a": 50, "warehouse_b": 30}}  
}  
  
orders = [  
    {"product": "laptop", "quantity": 3, "warehouse": "warehouse_a"},  
    {"product": "smartphone", "quantity": 5, "warehouse": "warehouse_b"},  
    {"product": "tablet", "quantity": 8, "warehouse": "warehouse_a"}  
]
```

Requirements:

- Process orders and update inventory
 - Calculate total revenue from processed orders
 - Identify low-stock items (total stock < 20)
 - Generate inventory report by warehouse
-

Task 4: Employee Attendance Tracker

Problem Statement: HR department needs to track employee attendance, calculate working hours, and identify attendance patterns.

Data:

python

```
attendance = {
    "John": [("2024-01-15", 9, 17), ("2024-01-16", 9.5, 18), ("2024-01-17", 8.5, 16.5)]
    "Sarah": [("2024-01-15", 8.5, 17.5), ("2024-01-16", 9, 17), ("2024-01-17", 9.5, 18)]
    "Mike": [("2024-01-15", 10, 16), ("2024-01-16", 9.5, 17.5), ("2024-01-17", 8, 15)]
}
# Format: (date, check_in_time, check_out_time)
```

Requirements:

- Calculate daily working hours for each employee
- Find average working hours per employee
- Identify employees working overtime (>8 hours/day)
- Create weekly attendance summary

Task 5: Restaurant Menu Optimizer

Problem Statement: A restaurant wants to analyze their menu performance, track popular dishes, and optimize pricing based on ingredients cost.

Data:

```
python

menu_items = {
    "Margherita Pizza": {"price": 450, "ingredients": ["dough", "tomato", "cheese"], "order": 1}
    "Chicken Burger": {"price": 350, "ingredients": ["bun", "chicken", "lettuce"], "order": 2}
    "Pasta Alfredo": {"price": 400, "ingredients": ["pasta", "cream", "cheese"], "order": 3}
    "Caesar Salad": {"price": 250, "ingredients": ["lettuce", "cheese", "croutons"], "order": 4}
}

ingredient_costs = {
    "dough": 50, "tomato": 30, "cheese": 80, "bun": 40,
    "chicken": 120, "lettuce": 20, "pasta": 60, "cream": 70, "croutons": 15
}
```

Requirements:

- Calculate profit margin for each dish
- Find most and least popular items
- Suggest price adjustments based on ingredient costs
- Create a performance report

Task 6: Library Book Management

Problem Statement: A library needs to manage book inventory, track borrowing patterns, and handle overdue books.

Data:

```
python

books = [
    {"id": 1, "title": "Python Programming", "author": "John Smith", "copies": 5, "borrowed": 0},
    {"id": 2, "title": "Data Science Basics", "author": "Jane Doe", "copies": 3, "borrowed": 1},
    {"id": 3, "title": "Web Development", "author": "Bob Johnson", "copies": 4, "borrowed": 2}
]

borrowers = {
    "Alice": [(1, "2024-01-10"), (2, "2024-01-15")],
    "Bob": [(1, "2024-01-12"), (3, "2024-01-18")],
    "Charlie": [(2, "2024-01-14"), (3, "2024-01-20")]
}

# Format: (book_id, borrow_date)
```

Requirements:

- Find available books for borrowing
- Calculate overdue books (borrowed > 14 days ago)
- Generate borrower history report
- Identify most popular books

Task 7: Bank Transaction Analyzer

Problem Statement: A bank needs to analyze customer transactions, detect spending patterns, and flag unusual activities.

Data:

python

```
transactions = {
    "customer_001": [
        {"date": "2024-01-15", "type": "debit", "amount": 2500, "category": "groceries"},
        {"date": "2024-01-16", "type": "credit", "amount": 50000, "category": "salary"},
        {"date": "2024-01-17", "type": "debit", "amount": 15000, "category": "rent"}
    ],
    "customer_002": [
        {"date": "2024-01-15", "type": "debit", "amount": 5000, "category": "shopping"},
        {"date": "2024-01-16", "type": "debit", "amount": 1200, "category": "utilities"},
        {"date": "2024-01-17", "type": "credit", "amount": 3000, "category": "freelance"}
    ]
}
```

Requirements:

- Calculate monthly spending by category for each customer
- Find customers with high spending (>20,000 in debits)
- Identify unusual transactions (amount >10,000)
- Generate spending pattern report

Task 8: Sports Tournament Manager

Problem Statement: Organize a sports tournament with multiple teams, track match results, and calculate standings.

Data:

python

```
teams = ["Lions", "Tigers", "Eagles", "Sharks", "Panthers"]
matches = [
    {"team1": "Lions", "team2": "Tigers", "score1": 3, "score2": 1},
    {"team1": "Eagles", "team2": "Sharks", "score1": 2, "score2": 2},
    {"team1": "Panthers", "team2": "Lions", "score1": 1, "score2": 4},
    {"team1": "Tigers", "team2": "Eagles", "score1": 0, "score2": 3}
]
```

Requirements:

- Calculate points for each team (Win: 3 points, Draw: 1 point, Loss: 0 points)
- Generate league table sorted by points
- Find teams with highest and lowest goal differences

- Create match statistics summary
-

Task 9: Weather Data Analyzer

Problem Statement: A meteorological department needs to analyze weather data across different cities and identify patterns.

Data:

python

```
weather_data = {  
    "Mumbai": [  
        {"date": "2024-01-15", "temp": 28, "humidity": 75, "rainfall": 0},  
        {"date": "2024-01-16", "temp": 30, "humidity": 80, "rainfall": 5},  
        {"date": "2024-01-17", "temp": 26, "humidity": 85, "rainfall": 15}  
    ],  
    "Delhi": [  
        {"date": "2024-01-15", "temp": 15, "humidity": 60, "rainfall": 0},  
        {"date": "2024-01-16", "temp": 18, "humidity": 55, "rainfall": 0},  
        {"date": "2024-01-17", "temp": 12, "humidity": 70, "rainfall": 2}  
    ]  
}
```

Requirements:

- Calculate average temperature and humidity for each city
 - Find days with rainfall for each city
 - Identify the hottest and coldest days across all cities
 - Generate weather summary report
-

Task 10: Online Course Progress Tracker

Problem Statement: An online learning platform needs to track student progress across multiple courses and generate completion reports.

Data:

python

```
students = {
    "student_001": {
        "name": "Alice Johnson",
        "courses": {
            "Python Basics": {"total_lessons": 20, "completed": 18, "quiz_scores": [85, 78, 90]},
            "Data Analysis": {"total_lessons": 15, "completed": 10, "quiz_scores": [92, 88, 80]}
        }
    },
    "student_002": {
        "name": "Bob Smith",
        "courses": {
            "Python Basics": {"total_lessons": 20, "completed": 20, "quiz_scores": [95, 90, 88]},
            "Web Development": {"total_lessons": 25, "completed": 22, "quiz_scores": [85, 82, 75]}
        }
    }
}
```

Requirements:

- Calculate completion percentage for each course per student
- Find average quiz scores for each student per course
- Identify students who completed all enrolled courses
- Generate progress report with recommendations

Practice Tips:

1. Use nested loops for complex data processing
2. Combine lists, tuples, dictionaries, and sets appropriately
3. Implement error handling for edge cases
4. Focus on code readability and efficiency
5. Test your solutions with different data scenarios